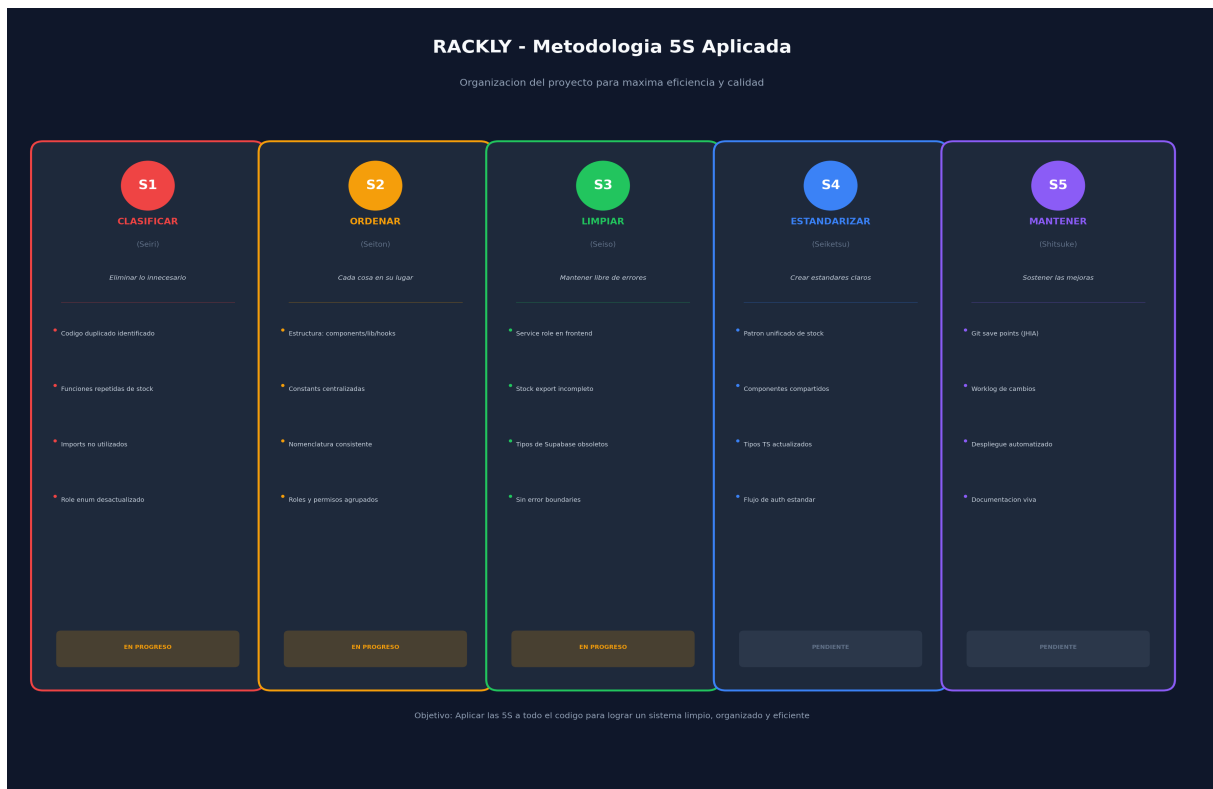


RACKLY

Metodologia 5S Aplicada al Proyecto



RACKLY_Mapa_5S_Resumen.png

Proyecto: RACKLY - Gestion de Almacen
Version: 2.0
Fecha: 25 de Mayo, 2026
Tecnologia: Next.js 16 + Supabase + Cloudflare Pages
URL Produccion: <https://rackly.pages.dev>
Metodologia: 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke)

Contenido

1. Introduccion a la Metodologia 5S
2. Estado Actual del Proyecto
3. S1 - CLASIFICAR (Seiri)
4. S2 - ORDENAR (Seiton)
5. S3 - LIMPIAR (Seiso)
6. S4 - ESTANDARIZAR (Seiketsu)
7. S5 - MANTENER (Shitsuke)
8. Mapa de Arquitectura del Sistema
9. Mapa de Flujo de Datos
10. Mapa de Integracion de Modulos
11. Plan de Accion y Prioridades
12. Conclusiones

1. Introduccion a la Metodologia 5S

La metodologia 5S es una tecnica japonesa de gestion de calidad desarrollada por Toyota como parte del Sistema de Produccion Toyota (TPS). Su objetivo es crear y mantener un entorno de trabajo organizado, limpio y eficiente que permita maximizar la productividad y minimizar los desperdicios. Las cinco "S" provienen de terminos japoneses que representan cinco pasos fundamentales para la mejora continua.

En el contexto de desarrollo de software, la metodologia 5S se adapta para organizar el codigo fuente, la estructura de archivos, las dependencias, la documentacion y los procesos de despliegue. Un proyecto de software "limpio" segun 5S es aquel donde cada archivo tiene un proposito claro, no existe codigo muerto o redundante, las convenciones son consistentes, y el equipo puede mantener y escalar el sistema sin fricciones innecesarias.

Paso	Termino Japonés	Significado en Español	Aplicacion en Software
S1	Seiri	Clasificar	Eliminar codigo muerto, dependencias obsoletas, archivos no utilizados
S2	Seiton	Ordenar	Organizar estructura de carpetas, naming conventions, modulo cohesion
S3	Seiso	Limpiar	Corregir bugs, vulnerabilidades, inconsistencias en los tipos
S4	Seiketsu	Estandarizar	Crear patrones reutilizables, actualizar documentacion, convenciones un
S5	Shitsuke	Mantener	Disciplina en el equipo, code review, save points, documentacion viva

Este documento aplica la metodologia 5S al proyecto RACKLY, un sistema de gestion de almacen desarrollado con Next.js 16, Supabase y desplegado en Cloudflare Pages. El analisis abarca tanto la estructura del codigo como la arquitectura de datos, los flujos de informacion y los mapas de integracion entre modulos. El objetivo final es que RACKLY funcione de la mejor manera posible, con un codigo limpio, organizado y facil de mantener y escalar.

2. Estado Actual del Proyecto

RACKLY es un sistema completo de gestion de almacen que actualmente cuenta con mas de 7,000 lineas de codigo distribuidas en 39 archivos fuente. La aplicacion se divide en dos vistas principales: la vista de Racks (con 8 modulos funcionales) y la vista de Piso (con 4 modulos). Utiliza Supabase como backend, con 13 tablas de base de datos y 5 funciones RPC (Remote Procedure Call) para operaciones complejas.

2.1 Estructura Tecnica General

Componente	Tecnologia	Version	Proposito
Frontend	Next.js	16.1.3	Framework React con exportacion estatica
Estilos	Tailwind CSS	4.x	Framework de utilidades CSS
UI Components	shadcn/ui	-	Componentes pre-construidos
Backend	Supabase	Cloud	Base de datos + Auth + Realtime
Base de Datos	PostgreSQL	15.x	13 tablas, 5 funciones RPC
Hosting	Cloudflare Pages	-	CDN global, deploy desde Git
Lenguaje	TypeScript	5.x	Tipado estatico
Control de Versiones	Git + GitHub	-	14 save points (JHIA1-JHIA11.14)

2.2 Modulos Implementados

La aplicacion cuenta con 12 modulos funcionales divididos en dos vistas. La vista de Racks gestiona las operaciones de almacenaje vertical (bloques con torres, pisos y posiciones), mientras que la vista de Piso gestiona las operaciones a nivel de suelo (sectores con columnas, subcolumnas y niveles). Cada modulo tiene responsabilidades especificas y se conecta con las tablas de base de datos correspondientes.

Vista	Modulo	Componente	Tablas Usadas	Lineas
Racks	Movimientos	MovimientoForm.tsx	movimientos, catalogo	902
Racks	Traslado	TrasladoTab.tsx	movimientos, catalogo	720
Racks	Catalogo	CatalogoTab.tsx	catalogo	429
Racks	Stock	StockTab.tsx	movimientos, catalogo	282
Racks	Ocupacion	OcupacionTab.tsx	movimientos, catalogo	~700
Racks	Descarga	DescargaTab.tsx	movimientos	599
Racks	FEFO	FefoTab.tsx	movimientos	312
Racks	Usuarios	UsuariosTab.tsx	profiles, user_roles	247
Piso	Movimientos	MovimientosTab.tsx	piso_movimientos, piso_mov_definiciones	624
Piso	UP Kardex	UpKardexTab.tsx	piso_bloques	163
Piso	Sectores	SectoresTab.tsx	piso_sectores	151
Piso	Config Columnas	ConfiguracionColumnasTab.tsx	piso_sectores, piso_columnas, piso_bloques	245

3. S1 - CLASIFICAR (Seiri)

El primer paso de la metodologia 5S consiste en identificar y clasificar todos los elementos del proyecto, diferenciando entre lo que es necesario y lo que es innecesario. En un proyecto de software, esto significa identificar codigo muerto, funciones duplicadas, dependencias no utilizadas, archivos huérfanos y cualquier elemento que no aporte valor funcional al sistema. El objetivo es mantener solo lo que es esencial para el funcionamiento correcto de la aplicacion.

3.1 Codigo Duplicado Identificado

Durante el analisis exhaustivo del codebase, se identificaron multiples instancias de logica duplicada que aumentan la complejidad del mantenimiento y crean riesgo de inconsistencias. A continuacion se detallan las principales duplicaciones encontradas:

Calculo de Stock: La logica de agregar movimientos para calcular stock por ubicacion esta re-implementada de forma independiente en al menos 5 componentes diferentes: StockTab.tsx, TrasladoTab.tsx, FefoTab.tsx, DescargaTab.tsx (incompleto) y OcupacionTab.tsx. Cada componente tiene su propia version del calculo, lo que significa que un cambio en la logica de stock requiere actualizar 5 archivos diferentes. Ademas, DescargaTab.tsx ignora los tipos devolucion y traslado en su calculo, lo cual genera exportaciones de Excel con datos incorrectos.

Funciones de Utilidad: Las funciones fmtCantidad() y formatDateTime() estan definidas tanto en utils.ts como en page.tsx (lineas 71-79). Esto crea confusion sobre cual version debe usarse y puede llevar a comportamientos inconsistentes si alguna version se modifica sin actualizar la otra.

Alerta de Ubicacion Ocupada: Tanto MovimientoForm.tsx (IngresoForm) como TrasladoTab.tsx tienen implementaciones casi identicas del dialogo "ubicacion ocupada" con el boton "Dar Salida". Este patron repetido deberia extraerse en un componente compartido reutilizable.

Parseo de Excel: La logica de auto-deteccion de columnas en archivos Excel esta duplicada en DescargaTab.tsx (lineas 178-207) y UpKardexTab.tsx (lineas 48-56), con la misma logica basada en expresiones regulares. Una funcion compartida eliminaria esta redundancia.

3.2 Elementos Obsoletos o No Utilizados

Se identificaron los siguientes elementos que no aportan valor funcional o que estan desactualizados respecto al estado actual del sistema:

Elemento	Ubicacion	Estado	Accion Recomendada
has_role()	types.ts (declarado)	Declarado pero nunca llamado	Eliminar de types.ts
primer_nombre_usuario()	types.ts (declarado)	Declarado pero nunca llamado	Eliminar de types.ts
Enum app_role	types.ts (linea 408)	Solo permite admin/operario, la App usa 7 roles	Actualizar roles en Supabase
Tipos catalogo	types.ts	No incluye stock_big_magic	Regenerar tipos con supabase gen
Error Boundaries	Toda la app	No existen React Error Boundaries	Agregar en page.tsx

3.3 Resultado de la Clasificacion

Tras aplicar Seiri, se identificaron 8 areas de codigo duplicado, 5 elementos obsoletos y 1 vulnerabilidad critica de seguridad. La siguiente tabla resume el inventario completo de elementos clasificados:

Categoria	Necesario	Duplicado	Obsoleto	Vulnerable
Funciones de negocio	25 funciones	5 duplicadas	2 no usadas	1 critica
Componentes UI	12 tabs + auth	2 patrones duplicados	0	0
Constantes	30 definidas	0	1 (POLLING sin uso)	0
Tipos TypeScript	14 interfaces	0	3 desactualizados	1 mismatch
Librerias	7 archivos .ts	0	0	1 service_role expuesto

4. S2 - ORDENAR (Seiton)

Seiton consiste en organizar los elementos necesarios de manera que cada uno tenga un lugar definido y sea facil de encontrar y usar. En desarrollo de software, esto se traduce en mantener una estructura de carpetas coherente, seguir convenciones de nomenclatura consistentes, y asegurar que cada modulo tenga una responsabilidad unica y bien definida (principio de cohesion). La estructura actual de RACKLY sigue buenas practicas en su organizacion general, pero hay areas especificas que pueden mejorarse.

4.1 Estructura de Carpetas Actual

RACKLY utiliza una estructura basada en el patron de Next.js App Router, con los componentes organizados por dominio funcional (kardex, piso, auth, ui). Las librerias de logica de negocio estan separadas de los componentes de presentacion, lo cual facilita el mantenimiento y las pruebas. La estructura actual es la siguiente:

Directorio	Contenido	Archivos	Evaluacion
src/app/	Paginas de la app	page.tsx, layout.tsx	Buena - minimo de archivos
src/components/rackly/kardex/	Vista Racks (8 tabs)	10 archivos .tsx	Buena - 1 archivo por modulo
src/components/rackly/piso/	Vista Piso (4 tabs)	4 archivos .tsx	Buena - 1 archivo por modulo
src/components/rackly/auth/	Autenticacion	AuthGate.tsx	Aceptable - archivo grande (667 lineas)
src/components/ui/	shadcn/ui base	13 archivos	Buena - sin modificaciones
src/lib/rackly/	Logica Racks	6 archivos .ts	Buena - separacion por dominio
src/lib/piso/	Logica Piso	1 archivo .ts	Aceptable - podria separarse
src/lib/supabase/	Cliente y tipos	2 archivos	Buena - estandar de Supabase
src/hooks/	Hooks personalizados	2 archivos	Buena - auth + realtime

4.2 Nomenclatura y Convenciones

El proyecto mantiene convenciones de nomenclatura consistentes en la mayoría de los archivos. Los componentes React usan PascalCase (MovimientoForm, StockTab), las funciones usan camelCase (fetchMovimientos, addMovimiento), y las constantes usan UPPER_SNAKE_CASE (BLOQUES, PISOS, ROL_ADMIN). Sin embargo, se identificaron algunas inconsistencias menores que podrian mejorarse para mantener un estandar perfecto.

4.3 Organizacion de Responsabilidades

Cada componente tiene una responsabilidad principal bien definida. Sin embargo, algunos archivos son demasiado grandes y contienen multiples sub-componentes internos que podrian extraerse como archivos independientes. Los archivos mas grandes son: OcupacionTab.tsx (~700 lineas), MovimientoForm.tsx (902 lineas) y AuthGate.tsx (667 lineas). Extraer sub-componentes mejoraria la legibilidad y facilitaria las pruebas unitarias.

4.4 Propuesta de Mejoras en Orden

- Extraer `LocationOccupiedDialog` como componente compartido (usado en `MovimientoForm` y `TrasladoTab`)
- Dividir `AuthGate.tsx` en archivos separados por cada screen (Login, Signup, Recovery, etc.)
- Extraer `IngresoForm` y `SalidaForm` desde `MovimientoForm.tsx` a archivos propios
- Separar `piso/api.ts` en archivos modulares (`sectores.ts`, `columnas.ts`, `movimientos.ts`)
- Crear un directorio `src/lib/rackly/stock/` para centralizar toda la logica de calculo de stock
- Unificar la cache de catalogo en un hook dedicado `useCatalogo()` en lugar de variables a nivel de modulo

5. S3 - LIMPIAR (Seiso)

Seiso significa limpiar y mantener el entorno de trabajo libre de errores, bugs, vulnerabilidades y cualquier tipo de suciedad que afecte el rendimiento del sistema. En el contexto de desarrollo de software, "limpiar" implica corregir bugs, parchear vulnerabilidades de seguridad, actualizar dependencias obsoletas, eliminar warnings del compilador y asegurar que el código compile sin errores.

5.1 Vulnerabilidades Críticas

VULNERABILIDAD CRITICA - Service Role Key Expuesto: Se ha detectado que la clave de servicio (`service_role`) de Supabase esta codificada directamente en el archivo `kardex.ts` (líneas 263-264 y 324-325), el cual se ejecuta en el navegador del cliente. Esta clave tiene acceso total a la base de datos, ignorando todas las políticas de Row Level Security (RLS). Cualquier persona que inspeccione el código del navegador puede extraer esta clave y obtener acceso completo a la base de datos, incluyendo la posibilidad de eliminar todos los datos, modificar perfiles de usuario y crear cuentas administrativas.

La solución recomendada es mover estas operaciones a un Supabase Edge Function o a una API Route del lado del servidor, donde la clave `service_role` este protegida como variable de entorno y nunca sea expuesta al navegador del cliente. Alternativamente, se puede crear una función RPC en PostgreSQL con `SECURITY DEFINER` que ejecute las operaciones con privilegios elevados sin exponer la clave.

5.2 Errores Funcionales

Exportacion de Stock Incompleta: El módulo `DescargaTab.tsx`, en su función de descarga de Excel, calcula el stock considerando únicamente los tipos "ingreso" y "salida", ignorando "devolucion" y "traslado". Esto genera que los reportes exportados muestren cantidades de stock incorrectas. Otros módulos como `StockTab` y `FefoTab` si incluyen todos los tipos de movimiento correctamente, lo cual crea una inconsistencia entre los datos mostrados en pantalla y los datos exportados.

Mismatch de Roles en Base de Datos: El enum `app_role` en la base de datos solo permite los valores "admin" y "operario", pero la aplicación define 7 roles diferentes (admin, operario, auxiliar, almacenero, supervisor_almacen, supervisor_operaciones, coordinador_operaciones). Cuando se intenta asignar un rol que no existe en el enum, la operación falla silenciosamente. Esto significa que las funciones de cambio de rol en `UsuariosTab.tsx` pueden no funcionar correctamente para la mayoría de los roles.

5.3 Tipos TypeScript Desactualizados

El archivo `types.ts` generado por Supabase presenta varias discrepancias con el esquema real de la base de datos. La columna `stock_big_magic` de la tabla `catalogo` no esta reflejada en los tipos generados, a pesar de que la aplicación la lee y escribe activamente. El enum de roles (`app_role`) solo contempla dos valores cuando la aplicación requiere siete. Es necesario regenerar los tipos ejecutando el comando de generación de Supabase y luego actualizar manualmente las discrepancias que no se resuelven automáticamente.

5.4 Ausencia de Manejo de Errores

6. S4 - ESTANDARIZAR (Seiketsu)

Seiketsu implica crear estandares y normas que mantengan los logros obtenidos en las tres primeras "S". En desarrollo de software, esto significa documentar las convenciones del proyecto, crear patrones reutilizables, establecer guias de estilo de codigo y automatizar las verificaciones de calidad. Los estandares deben ser claros, accesibles y faciles de seguir por cualquier desarrollador que se incorpore al proyecto.

6.1 Patron Unificado de Calculo de Stock

El problema mas urgente a estandarizar es el calculo de stock. Actualmente, cada componente que necesita calcular stock implementa su propia version, con variaciones sutiles que producen inconsistencias. Se propone crear una funcion centralizada `calcularStockGeneral(movimientos)` en `utils.ts` que reciba la lista completa de movimientos y retorne un mapa de ubicaciones con su stock calculado. Esta funcion deberia incluir correctamente todos los tipos de movimiento: ingreso (+), salida (-), devolucion (+) y traslado (+ en destino, - en origen). Todos los componentes existentes deberian migrar a usar esta funcion unica.

6.2 Patron de Componentes Compartidos

Se recomienda extraer los patrones repetidos en componentes compartidos reutilizables. Esto incluye: `LocationOccupiedDialog` (dialogo de ubicacion ocupada con opcion de salida), `ExcelUploadPanel` (panel de subida de archivos Excel con preview), `StockCard` (tarjeta de resumen de stock por ubicacion), y `MovementHistoryTable` (tabla de historial de movimientos con filtros). Estos componentes compartidos reducen la duplicacion de codigo y aseguran consistencia visual y funcional en toda la aplicacion.

6.3 Convenciones de Comunicacion con Supabase

Se recomienda estandarizar la forma en que los componentes interactuan con Supabase. Actualmente, algunos componentes llaman directamente a `supabase.from().select()` mientras que otros usan funciones de la capa de librerias (`kardex.ts`, `auth.ts`, etc.). La regla deberia ser: los componentes NUNCA deberian importar o usar directamente el cliente de Supabase; toda comunicacion con la base de datos deberia pasar por las funciones de las librerias correspondientes. Esto facilita el testing, permite cambiar la implementacion sin afectar los componentes, y centraliza el manejo de errores.

6.4 Guia de Roles y Permisos

Rol	Acceso UP	Eliminar Mov	Aprobar Us	Cambiar Rol	Eliminar Perfil
admin	Si	Si	Si	Si	Si
coordinador_operaciones	Si	Si	Si	No	No
supervisor_almacen	No	Si	Si	No	No
supervisor_operaciones	No	Si	Si	No	No
almacenero	No	No	No	No	No
auxiliar	No	No	No	No	No

Rol	Acceso UP	Eliminar Mod	Aprobar Us	Eliminar Rol	Eliminar Perfil
operario	No	No	No	No	No

7. S5 - MANTENER (Shitsuke)

Shitsuke es la disciplina de mantener los logros obtenidos con las cuatro "S" anteriores y convertirlos en habitos permanentes. En desarrollo de software, esto se traduce en mantener la disciplina de code review, seguir las convenciones establecidas, documentar los cambios, mantener los save points y asegurar que cada despliegue sea un proceso controlado y verificable. Sin disciplina, las mejoras obtenidas se deterioran con el tiempo y el proyecto vuelve a acumular deuda tecnica.

7.1 Sistema de Save Points (JHIA)

RACKLY utiliza un sistema de save points basados en Git que permite rastrear la evolucion del proyecto y revertir cambios si es necesario. Cada save point corresponde a un commit en Git con un prefijo JHIA seguido de un numero secuencial. Este sistema ha demostrado ser efectivo para mantener un historial claro de los cambios y facilitar la colaboracion.

Save Point	Descripcion	Elementos Clave
JHIA1-JHIA7	Fundacion del proyecto	Auth, DB, Estructura base, Tema oscuro
JHIA8-JHIA9	Modulos avanzados	Ocupacion 3D, FEFO, Filtros, Tema oscuro completo
JHIA10-JHIA11	Integracion de datos	UP Data Excel, Batch 1000, Service role, Permisos
JHIA11.12-JHIA11.13	Seguridad y permisos	Restriccion UP Data, Roles granulares
JHIA11.14	Mejoras visuales	Colores Ocupacion: azul/verde/naranja

7.2 Proceso de Despliegue

El flujo de despliegue actual sigue el patron: desarrollo local con npm run dev, verificacion con npm run build, commit con save point, push a GitHub, y deploy manual via Wrangler CLI a Cloudflare Pages. Este proceso ha funcionado de manera confiable y deberia mantenerse como estandar. Se recomienda documentar este proceso en un archivo README dedicado y considerar la automatizacion del deploy mediante GitHub Actions para eliminar la necesidad de deploy manual.

7.3 Worklog de Cambios

Se ha implementado un sistema de worklog en /home/z/my-project/worklog.md donde cada agente que trabaja en el proyecto registra las acciones realizadas, los resultados obtenidos y las decisiones tomadas. Este registro permite reconstruir el historial de decisiones y facilita la transferencia de conocimiento entre sesiones de desarrollo. El formato estandarizado del worklog incluye: Task ID, Agent, Task, Work Log y Stage Summary.

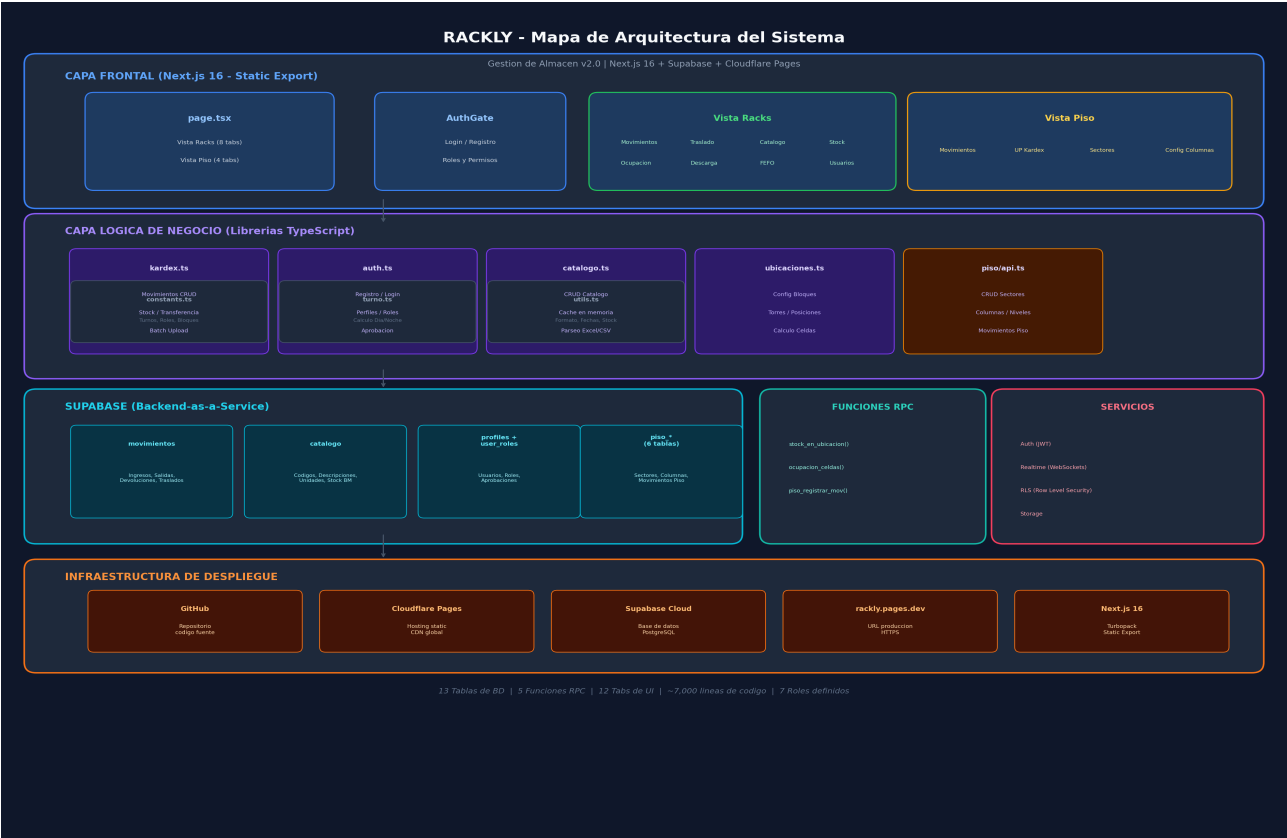
7.4 Checklist de Mantenimiento Periodico

- Verificar que npm run build compile sin errores ni warnings
- Confirmar que el deploy en Cloudflare Pages se completo exitosamente
- Revisar que los tipos TypeScript estan sincronizados con el esquema de Supabase
- Verificar que no hay nuevas funciones duplicadas o codigo muerto

- Confirmar que las políticas RLS en Supabase siguen siendo efectivas
- Revisar el uso de la clave service_role y asegurar que no se exponga en nuevos archivos
- Actualizar el worklog con los cambios realizados en la sesión
- Crear un nuevo save point (JHIA) para cada conjunto de cambios significativos

8. Mapa de Arquitectura del Sistema

El siguiente mapa muestra la arquitectura completa de RACKLY, organizada en cuatro capas: Frontend (Next.js 16), Logica de Negocio (librerias TypeScript), Base de Datos (Supabase con PostgreSQL) e Infraestructura de Despliegue (Cloudflare Pages). Cada capa esta representada con sus componentes principales y las relaciones entre ellos.



RACKLY_Mapa_Arquitectura.png

9. Mapa de Flujo de Datos

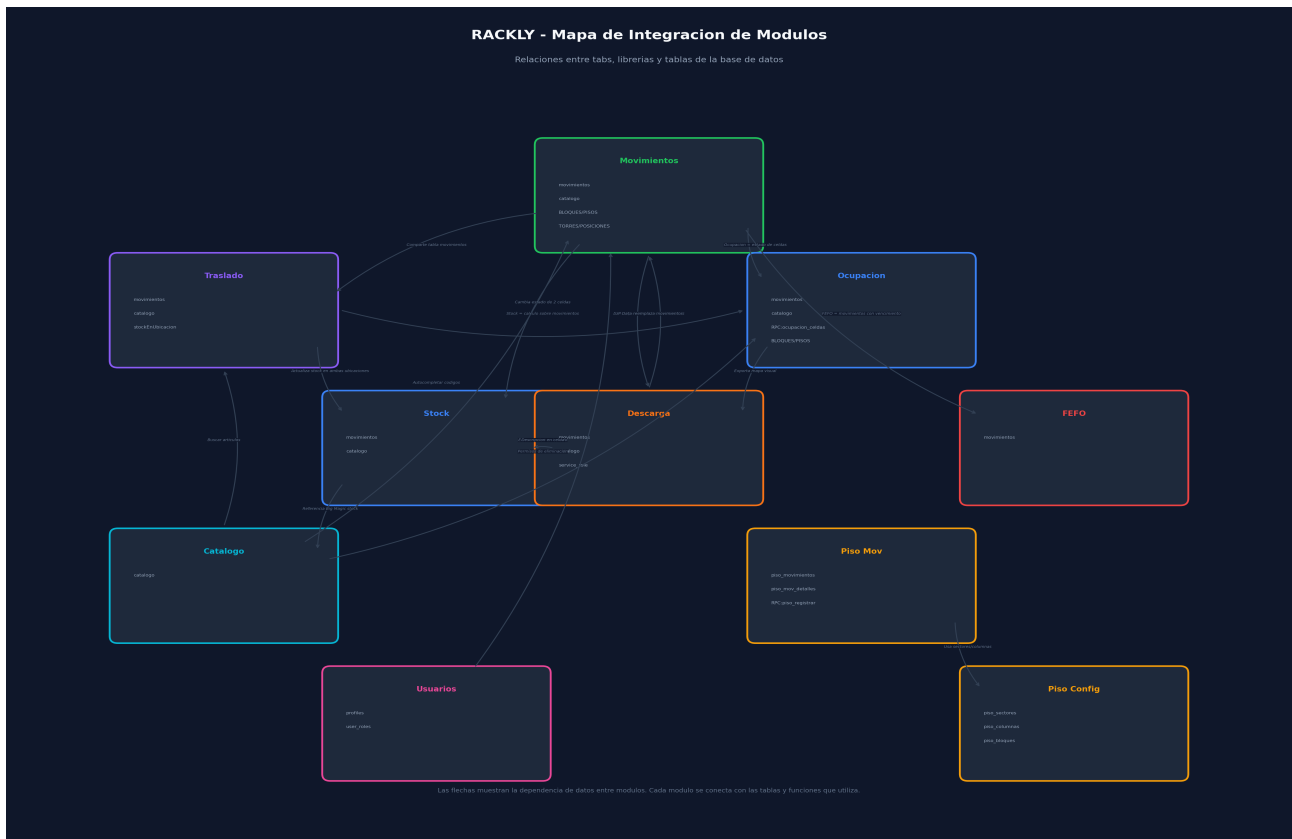
Este mapa ilustra como fluye la informacion desde las acciones del usuario, pasando por la capa de procesamiento (validaciones, funciones RPC, logica de stock), hasta la base de datos y finalmente a las vistas que se actualizan. Las flechas muestran las operaciones SQL realizadas (INSERT, SELECT, RPC, BATCH) y la sincronizacion en tiempo real via WebSockets.



RACKLY_Mapa_FlujoDatos.png

10. Mapa de Integracion de Modulos

El mapa de integracion muestra las dependencias de datos entre los 10 modulos principales de RACKLY. Cada modulo se conecta con las tablas y funciones RPC que utiliza. Las flechas indican la direccion de la dependencia y las etiquetas describen que tipo de datos se comparten. Este mapa es esencial para entender el impacto de cambios en un modulo sobre los demas.



RACKLY_Map_Integracion.png

11. Plan de Accion y Prioridades

A continuacion se presenta el plan de accion priorizado para aplicar las 5S al proyecto RACKLY. Las acciones se organizan por prioridad: critica (debe hacerse inmediatamente), alta (proxima sesion), media (a mediano plazo) y baja (mejora continua). Cada accion incluye el paso 5S al que pertenece, una descripcion breve y los archivos afectados.

Prioridad	Accion	Paso 5S	Archivos Afectados
CRITICA	Mover service_role a Edge Function o RPT	S3	kardex.ts
CRITICA	Actualizar enum app_role en Supabase	S3	types.ts, auth.ts, Supabase DB
CRITICA	Corregir calculo de stock en DescargaTab	S1b	DescargaTab.tsx
ALTA	Crear calcularStockGeneral() centralizado	S4b	utils.ts, 5 componentes
ALTA	Extraer LocationOccupiedDialog compartido	S2b	MovimientoForm, TrasladoTab
ALTA	Agregar Error Boundaries	S3	page.tsx
ALTA	Regenerar tipos Supabase	S3	types.ts
ALTA	Eliminar funciones no usadas (has_role, etc.)	S3	types.ts
MEDIA	Dividir AuthGate.tsx en sub-componentes	S2	AuthGate.tsx
MEDIA	Dividir piso/api.ts en modulos	S2	piso/api.ts
MEDIA	Eliminar duplicacion fmtCantidad/formaDateTime	S2	page.tsx, utils.ts
MEDIA	Extraer IngresoForm y SalidaForm	S2	MovimientoForm.tsx
MEDIA	Unificar cache de catalogo en hook	S4	catalogo.ts, nuevo hook
BAJA	Automatizar deploy con GitHub Actions	S5	CI/CD config
BAJA	Crear README con guias de desarrollo	S5	README.md
BAJA	Agregar tests unitarios para utils	S4	tests/
BAJA	Extraer ExcelUploadPanel compartido	S2	DescargaTab, UpKardexTab

12. Conclusiones

La aplicacion de la metodologia 5S al proyecto RACKLY ha revelado un sistema robusto y funcional con mas de 7,000 lineas de codigo, 12 modulos activos y 13 tablas de base de datos. La aplicacion cumple su objetivo principal de gestionar almacenes de manera eficiente, con funcionalidades que cubren desde el registro de movimientos basicos hasta el control FEFO, la visualizacion 3D de ocupacion y la carga masiva de datos via Excel.

Sin embargo, el analisis tambien ha identificado areas significativas de mejora. La vulnerabilidad critica del service role expuesto en el frontend requiere atencion inmediata, al igual que la inconsistencia en el calculo de stock en el modulo de descarga. Las duplicaciones de codigo, aunque funcionales, aumentan el riesgo de errores futuros y dificultan el mantenimiento. Los tipos TypeScript desactualizados pueden causar bugs sutiles que son dificiles de detectar.

Los mapas de arquitectura, flujo de datos e integracion generados como parte de este analisis constituyen una herramienta valiosa para el equipo. Proporcionan una vision completa de como estan conectados los diferentes modulos, que tablas de la base de datos utiliza cada componente y como fluye la informacion a traves del sistema. Estos mapas deberian mantenerse actualizados a medida que el proyecto evoluciona.

La implementacion del plan de accion priorizado, comenzando por las acciones criticas y avanzando progresivamente hacia las mejoras a mediano y largo plazo, permitira que RACKLY alcance su maximo potencial como herramienta de gestion de almacen. La disciplina de mantener las 5S como practica permanente asegurara que el sistema continue siendo limpio, organizado y eficiente a medida que crece en funcionalidades y usuarios.